

4. Où vit une variable : avant la boucle, ou dedans

Une variable déclarée **DANS** la boucle renaît à chaque tour — et oublie tout.

1. Ce qu'il faut comprendre

Une variable existe à partir de sa déclaration, et **jusqu'à l'accolade qui ferme son bloc**. C'est sa **portée**. Hors de ses accolades, son nom ne veut plus rien dire.

`tours` est déclarée **avant** `while (true)` : elle naît une seule fois, et garde sa valeur d'un tour à l'autre. Elle compte : 1, 2, 3...

`neuve` est déclarée **dans** la boucle : à chaque tour, la ligne `uint8_t neuve = 0;` est exécutée de nouveau, et remet la case à zéro. On a beau faire `neuve++`, elle vaut toujours **1** à l'écran.

C'est **le bogue le plus fréquent du débutant** : « mon compteur ne compte pas ». Règle simple : **ce qui doit durer d'une image à l'autre** (un score, une position, une vie) se déclare **avant** la boucle du jeu. Ce qui ne sert que le temps d'un calcul se déclare dedans — c'est même plus clair, car on voit qu'elle ne sert qu'ici.

À toi : déplace la déclaration de `neuve` au-dessus de `while (true)`, et regarde-la compter à son tour.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **710 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         65 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
EcrireNombre   33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles   76 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles        368 o ← CopierTuiles - 23 lettres sur 43
"AVANT LA BOUCLE:" 16 o ← texte() - un texte gravé
"DANS LA BOUCLE:"  15 o ← texte() - un texte gravé
```

Lettres gravées : ABCDELNOSTUV0123456789:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 2    22 o sur place, dans main() + EcrireTexte (10 o)
nombre() × 2   20 o sur place, dans main() + EcrireNombre (33 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : compare deux variables.  
// - « tours » est créée AVANT la boucle : elle garde sa valeur d'un tour à l'autre.  
// - « neuve » est créée DANS la boucle : elle est recréée à 0 à chaque tour.  
  
#include <texte>    // écrit un texte à l'écran  
#include <nombre>   // écrit un nombre en chiffres
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

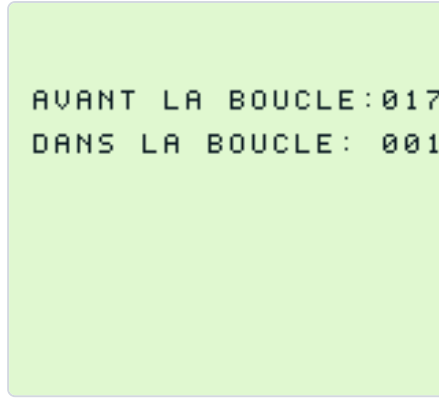
```
int main() {                // Le jeu commence ici.  
    uint8_t tours = 0;      // Crée UNE seule fois, avant la boucle : elle se souvient.  
  
    texte(1, 4, "AVANT LA BOUCLE:"); // Le nom de la première ligne.  
    texte(1, 6, "DANS LA BOUCLE:");  // Le nom de la deuxième ligne.  
  
    while (true) {          // Répète pour toujours ce qui est entre les accolades.  
        image();            // Attend l'image suivante.  
  
        uint8_t neuve = 0;   // Recrée à CHAQUE tour, et remise à 0 : elle oublie.  
        tours++;            // tours gagne 1 : 1, 2, 3, 4 ... (elle monte)  
        neuve++;            // neuve gagne 1 : elle passe de 0 à 1, à chaque tour.  
  
        nombre(17, 4, tours); // Affiche tours : le nombre grandit sans arrêt.  
        nombre(17, 6, neuve); // Affiche neuve : toujours 001.  
    }  
}
```

3. Ce qui se passe, étape par étape

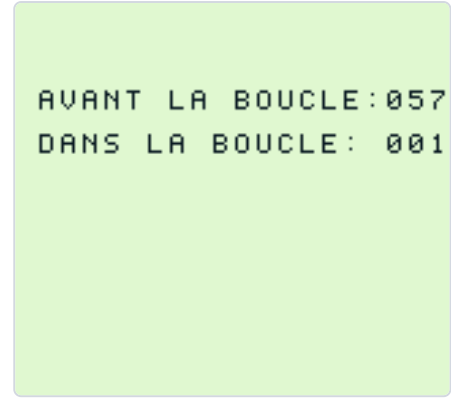
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



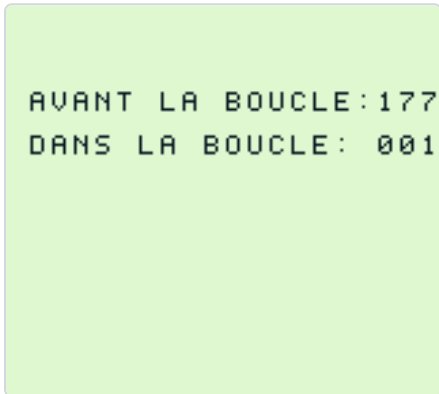
la toute première image — la console vient de démarrer — image 1



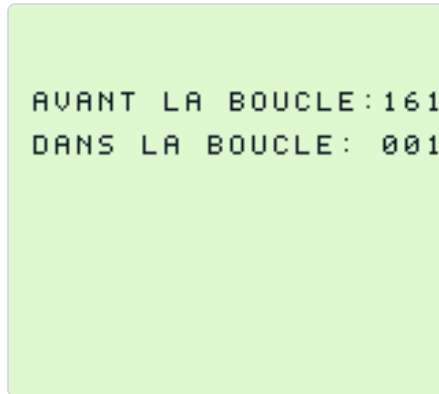
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le compteur du haut monte ; celui du bas reste bloqué sur 001.

5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la variable déclarée avant la boucle compte (17 puis 47)

✓ celle déclarée dedans vaut toujours 1

← [3. Un octet s'arrête à 255](#)

[sommaire](#)

[5. if et else : deux chemins](#) →